МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Навчально-науковий інститут комп'ютерних наук та інформаційних технологій
Кафедра автоматизованих систем управління

ЗВІТ

про виконання лабораторної роботи №9

«Теми оформлення та робота з ресурсами застосунку. Створення екрана профілю»

з дисципліни «Програмування мобільних додатків»

Виконав:
студент групи КН-2327Б
Багрій-Белз Андрій Володимирович

Перевірив:
доцент кафедри АСУ

Львів – 2026

1. Мета та завдання лабораторної роботи

Мета роботи: Ознайомитися з можливостями оформлення Flutter-застосунку, навчитися використовувати теми (Light / Dark mode), працювати з ресурсами (шрифти, зображення, іконки), організовувати стиль проєкту та створити повнофункціональний екран профілю користувача ProfileScreen з підтримкою динамічного перемикання теми.

Завдання: Перелік обов'язкових завдань згідно з навчально-методичною програмою:

- 1. Реалізувати підтримку світлої та темної теми (Light / Dark mode).
- 2. Додати перемикач теми (Switch або SwitchListTile).
- 3. Підключити власний шрифт (Inter) до проєкту через pubspec.yaml.
- 4. Додати графічні ресурси (банер, аватар користувача в assets/images/).
- 5. Використати стандартні та власні іконки (assets/icons/ у форматі PNG).
- 6. Створити екран ProfileScreen (аватар, ім'я, email, статус, перемикач теми, іконки оформлення).
- 7. Створити файл AppColors та додати власну палітру кольорів (primary, secondary, background, text).
- 8. Використовувати AppColors у всьому застосунку замість жорстко закодованих стандартних кольорів.
- 9. Створити файл AppStyles та визначити стилі тексту (title, subtitle, body, button, caption).
- 10. Використовувати AppStyles у ProfileScreen та інших компонентах.
- 11. Забезпечити єдиний стиль оформлення застосунку (однакові кольори, шрифти, узгоджені відступи).
- 12. Перевірити коректну роботу застосунку у світлій та темній темах на емуляторі Android.

2. Архітектура та опис реалізованого рішення

У межах виконання лабораторної роботи розроблено мобільний застосунок profile_app на базі Flutter SDK та архітектурного підходу Material 3 з повною підтримкою динамічного перемикання світлої та темної тем, інтеграцією локальних ресурсів та централізованою дизайн-системою.

Структура проєкту: Файлова структура проєкту організована згідно з рекомендаціями навчального посібника:

- assets/fonts/Inter-Regular.ttf, Inter-SemiBold.ttf — підключені власні шрифти родини Inter;
- assets/images/avatar.png, banner.png — растрові графічні ресурси (аватар та художній банер екрана);
- assets/icons/profile.png, settings.png, theme.png, logout.png, shield.png, bell.png — власні графічні іконки;
- lib/theme/app_colors.dart — централізований репозиторій дизайн-токенів кольорів AppColors;
- lib/theme/app_styles.dart — централізований репозиторій типографічних стилів AppStyles на базі шрифту Inter;
- lib/theme/app_theme.dart — генератор конфігурацій ThemeData для lightTheme та darkTheme;
- lib/screens/profile_screen.dart — віджет екрана профілю користувача з повною верстkою та інтерактивними блоками;

- lib/main.dart — точка входу в програму, керування станом ThemeMode (StatefulWidget) та конфігурація MaterialApp;
- test/widget_test.dart — автоматизовані віджет-тести для перевірки рендерингу, перемикання теми та діалогів.

3. Лістинг коду програми

Лістинг: pubspec.yaml (Реєстрація ресурсів та шрифтів Inter)

```yaml
name: profile_app
description: "Лабораторна робота №9: Теми оформлення та робота з ресурсами застосунку."
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: ^3.13.2

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true

  assets:
    - assets/images/
    - assets/icons/

  fonts:
    - family: Inter
      fonts:
        - asset: assets/fonts/Inter-Regular.ttf
        - asset: assets/fonts/Inter-SemiBold.ttf
          weight: 600
```

Лістинг: lib/theme/app_colors.dart (Власна палітра кольорів AppColors)

```dart
import 'package:flutter/material.dart';

/// Власна палітра кольорів застосунку AppColors (п. 7 ТЗ)
/// Використовується замість стандартних кольорів (Colors.blue тощо, п. 8 ТЗ)
class AppColors {
  // Основні брендові кольори
  static const Color primary = Color(0xFF4A90E2);       // Елегантний сапфіровий синій
  static const Color primaryDark = Color(0xFF2A68B2);   // Темно-синій акцент
  static const Color secondary = Color(0xFFF5A623);     // Сонячний бурштиновий
  static const Color accent = Color(0xFFD4FF00);        // Неоновий лайм

  // Кольори фону для світлої та темної тем
  static const Color lightBackground = Color(0xFFF8F9FA); // Світло-сірий м'який фон
  static const Color darkBackground = Color(0xFF121212);  // Глибокий графітовий темний фон
  static const Color lightCard = Color(0xFFFFFFFF);       // Біла картка
  static const Color darkCard = Color(0xFF1E1E1E);        // Темна картка
  static const Color lightSurface = Color(0xFFEFF2F6);    // Поверхня для світлої теми
  static const Color darkSurface = Color(0xFF252932);     // Поверхня для темної теми
```

```
19
20    // Кольори тексту
21    static const Color textLight = Color(0xFF1A1D20);       // Основний текст (світла тема)
22    static const Color textDark = Color(0xFFFFFFFF);        // Основний текст (темна тема)
23    static const Color textSecondaryLight = Color(0xFF6C757D); // Вторинний текст (світла)
24    static const Color textSecondaryDark = Color(0xFFA0A6B2);  // Вторинний текст (темна)
25
26    // Додаткові кольори
27    static const Color grey = Color(0xFF9E9E9E);
28    static const Color dividerLight = Color(0xFFE2E8F0);
29    static const Color dividerDark = Color(0xFF2D3748);
30    static const Color success = Color(0xFF2ECC71);
31    static const Color error = Color(0xFFE53935);
32    static const Color info = Color(0xFF17A2B8);
33
34    // Динамічні селектори за ознакою теми
35    static Color background(bool isDark) => isDark ? darkBackground : lightBackground;
36    static Color card(bool isDark) => isDark ? darkCard : lightCard;
37    static Color text(bool isDark) => isDark ? textDark : textLight;
38    static Color textSecondary(bool isDark) => isDark ? textSecondaryDark : textSecondaryLight;
39    static Color divider(bool isDark) => isDark ? dividerDark : dividerLight;
40    static Color surface(bool isDark) => isDark ? darkSurface : lightSurface;
41  }
```

Лістинг: lib/theme/app_styles.dart (Уніфіковані стилі тексту AppStyles)

```
1   import 'package:flutter/material.dart';
2   import 'app_colors.dart';
3
4   /// Стандартизовані стилі тексту AppStyles (п. 9 ТЗ)
5   /// Використовують підключений власний шрифт Inter (п. 3 ТЗ)
6   class AppStyles {
7     // Назва підключеного шрифту
8     static const String fontFamily = 'Inter';
9
10    // 1. Заголовок (title)
11    static TextStyle title({bool isDark = false, Color? color}) {
12      return TextStyle(
13        fontFamily: fontFamily,
14        fontSize: 24,
15        fontWeight: FontWeight.bold,
16        letterSpacing: -0.5,
17        color: color ?? (isDark ? AppColors.textDark : AppColors.textLight),
18      );
19    }
20
21    // 2. Підзаголовок (subtitle)
22    static TextStyle subtitle({bool isDark = false, Color? color}) {
23      return TextStyle(
24        fontFamily: fontFamily,
25        fontSize: 15,
26        fontWeight: FontWeight.w500,
27        color: color ??
28            (isDark ? AppColors.textSecondaryDark : AppColors.textSecondaryLight),
29      );
30    }
31
32    // 3. Основний текст (body)
33    static TextStyle body({bool isDark = false, Color? color, FontWeight? fontWeight}) {
34      return TextStyle(
35        fontFamily: fontFamily,
36        fontSize: 14,
37        fontWeight: fontWeight ?? FontWeight.normal,
38        height: 1.45,
39        color: color ?? (isDark ? AppColors.textDark : AppColors.textLight),
```

```
    );
  }

  // 4. Стиль для кнопок (button)
  static const TextStyle button = TextStyle(
    fontFamily: fontFamily,
    fontSize: 15,
    fontWeight: FontWeight.w600,
    letterSpacing: 0.3,
    color: Colors.white,
  );

  // 5. Стиль підпису або дати / статусу (caption)
  static TextStyle caption({bool isDark = false, Color? color}) {
    return TextStyle(
      fontFamily: fontFamily,
      fontSize: 12,
      fontWeight: FontWeight.w500,
      color: color ??
          (isDark ? AppColors.textSecondaryDark : AppColors.textSecondaryLight),
    );
  }

  // 6. Стиль бейджа або плашки
  static const TextStyle badge = TextStyle(
    fontFamily: fontFamily,
    fontSize: 11,
    fontWeight: FontWeight.w700,
    letterSpacing: 0.5,
  );

  // Статичні константи для швидкого застосування без параметрів
  static const TextStyle titleStatic = TextStyle(
    fontFamily: fontFamily,
    fontSize: 24,
    fontWeight: FontWeight.bold,
    letterSpacing: -0.5,
  );

  static const TextStyle subtitleStatic = TextStyle(
    fontFamily: fontFamily,
    fontSize: 15,
    fontWeight: FontWeight.w500,
  );

  static const TextStyle bodyStatic = TextStyle(
    fontFamily: fontFamily,
    fontSize: 14,
    fontWeight: FontWeight.normal,
    height: 1.45,
  );
}
```

Лістинг: lib/theme/app_theme.dart (Конфігурація ThemeData light & dark)

```
import 'package:flutter/material.dart';
import 'app_colors.dart';
import 'app_styles.dart';

/// Конфігурація централізованих тем застосунку ThemeData (п. 1, 11 ТЗ)
class AppTheme {
  /// Світла тема застосунку
  static ThemeData get lightTheme {
    return ThemeData(
      useMaterial3: true,
```

```
      brightness: Brightness.light,
      fontFamily: AppStyles.fontFamily,
      scaffoldBackgroundColor: AppColors.lightBackground,
      colorScheme: const ColorScheme.light(
        primary: AppColors.primary,
        secondary: AppColors.secondary,
        surface: AppColors.lightCard,
        onSurface: AppColors.textLight,
        error: AppColors.error,
      ),
      appBarTheme: const AppBarTheme(
        backgroundColor: AppColors.lightCard,
        foregroundColor: AppColors.textLight,
        elevation: 0,
        centerTitle: true,
        iconTheme: IconThemeData(color: AppColors.textLight),
        titleTextStyle: TextStyle(
          fontFamily: AppStyles.fontFamily,
          fontSize: 20,
          fontWeight: FontWeight.bold,
          color: AppColors.textLight,
        ),
      ),
      cardTheme: CardThemeData(
        color: AppColors.lightCard,
        elevation: 2,
        shadowColor: Colors.black.withValues(alpha: 0.05),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(16),
          side: const BorderSide(color: AppColors.dividerLight),
        ),
      ),
      dividerColor: AppColors.dividerLight,
      elevatedButtonTheme: ElevatedButtonThemeData(
        style: ElevatedButton.styleFrom(
          backgroundColor: AppColors.primary,
          foregroundColor: Colors.white,
          textStyle: AppStyles.button,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(12),
          ),
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 14),
          elevation: 2,
        ),
      ),
      switchTheme: SwitchThemeData(
        thumbColor: WidgetStateProperty.resolveWith((states) {
          if (states.contains(WidgetState.selected)) {
            return AppColors.primary;
          }
          return AppColors.grey;
        }),
        trackColor: WidgetStateProperty.resolveWith((states) {
          if (states.contains(WidgetState.selected)) {
            return AppColors.primary.withValues(alpha: 0.4);
          }
          return AppColors.dividerLight;
        }),
      ),
    );
  }

  /// Темна тема застосунку
  static ThemeData get darkTheme {
    return ThemeData(
      useMaterial3: true,
      brightness: Brightness.dark,
      fontFamily: AppStyles.fontFamily,
```

```
      scaffoldBackgroundColor: AppColors.darkBackground,
      colorScheme: const ColorScheme.dark(
        primary: AppColors.primary,
        secondary: AppColors.secondary,
        surface: AppColors.darkCard,
        onSurface: AppColors.textDark,
        error: AppColors.error,
      ),
      appBarTheme: const AppBarTheme(
        backgroundColor: AppColors.darkCard,
        foregroundColor: AppColors.textDark,
        elevation: 0,
        centerTitle: true,
        iconTheme: IconThemeData(color: AppColors.textDark),
        titleTextStyle: TextStyle(
          fontFamily: AppStyles.fontFamily,
          fontSize: 20,
          fontWeight: FontWeight.bold,
          color: AppColors.textDark,
        ),
      ),
      cardTheme: CardThemeData(
        color: AppColors.darkCard,
        elevation: 0,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(16),
          side: const BorderSide(color: AppColors.dividerDark),
        ),
      ),
      dividerColor: AppColors.dividerDark,
      elevatedButtonTheme: ElevatedButtonThemeData(
        style: ElevatedButton.styleFrom(
          backgroundColor: AppColors.primary,
          foregroundColor: Colors.white,
          textStyle: AppStyles.button,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(12),
          ),
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 14),
          elevation: 2,
        ),
      ),
      switchTheme: SwitchThemeData(
        thumbColor: WidgetStateProperty.resolveWith((states) {
          if (states.contains(WidgetState.selected)) {
            return AppColors.secondary;
          }
          return AppColors.grey;
        }),
        trackColor: WidgetStateProperty.resolveWith((states) {
          if (states.contains(WidgetState.selected)) {
            return AppColors.secondary.withValues(alpha: 0.4);
          }
          return AppColors.dividerDark;
        }),
      ),
    );
  }
}
```

Лістинг: lib/main.dart (Керування ThemeMode та MaterialApp)

```
import 'package:flutter/material.dart';
import 'screens/profile_screen.dart';
import 'theme/app_theme.dart';
```

```dart

void main() {
  runApp(const ProfileApp());
}

/// Головний кореневий віджет застосунку ProfileApp
class ProfileApp extends StatefulWidget {
  const ProfileApp({super.key});

  @override
  State<ProfileApp> createState() => _ProfileAppState();
}

class _ProfileAppState extends State<ProfileApp> {
  // Стан активної теми: true — темна, false — світла (п. 1, 2 ТЗ)
  bool _isDarkMode = true;

  void _toggleTheme(bool isDark) {
    setState(() {
      _isDarkMode = isDark;
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Профіль та теми — Лабораторна №9',
      debugShowCheckedModeBanner: false,
      // Налаштування світлої та темної тем через ThemeData (п. 1, 11 ТЗ)
      theme: AppTheme.lightTheme,
      darkTheme: AppTheme.darkTheme,
      themeMode: _isDarkMode ? ThemeMode.dark : ThemeMode.light,
      home: ProfileScreen(
        isDarkMode: _isDarkMode,
        onToggleTheme: _toggleTheme,
      ),
    );
  }
}
```

Лістинг: lib/screens/profile_screen.dart (Екран профілю користувача)

```dart
import 'package:flutter/material.dart';
import '../theme/app_colors.dart';
import '../theme/app_styles.dart';

/// Екран профілю користувача ProfileScreen (п. 6 ТЗ)
class ProfileScreen extends StatelessWidget {
  /// Поточний стан теми (true — темна, false — світла)
  final bool isDarkMode;

  /// Колбек для перемикання теми (п. 2 ТЗ)
  final ValueChanged<bool> onToggleTheme;

  const ProfileScreen({
    super.key,
    required this.isDarkMode,
    required this.onToggleTheme,
  });

  @override
  Widget build(BuildContext context) {
    final isDark = isDarkMode;

    return Scaffold(
```

```
      backgroundColor: AppColors.background(isDark),
      appBar: AppBar(
        title: Text(
          'Профіль користувача',
          style: AppStyles.title(isDark: isDark).copyWith(fontSize: 20),
        ),
        centerTitle: true,
        backgroundColor: AppColors.card(isDark),
        elevation: 0,
        actions: [
          IconButton(
            tooltip: 'Налаштування',
            icon: Image.asset(
              'assets/icons/settings.png',
              width: 22,
              height: 22,
            ),
            onPressed: () {
              _showSnackBar(context, 'Відкрито налаштування профілю');
            },
          ),
          IconButton(
            tooltip: 'Вийти з облікового запису',
            icon: Image.asset(
              'assets/icons/logout.png',
              width: 22,
              height: 22,
            ),
            onPressed: () => _confirmLogout(context, isDark),
          ),
        ],
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.symmetric(horizontal: 16.0, vertical: 12.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              // 1. Картка аватара та основної інформації (п. 4, 6 ТЗ)
              _buildHeaderCard(context, isDark),

              const SizedBox(height: 16),

              // 2. Блок статистики та відомостей про навчання
              _buildStatsCard(isDark),

              const SizedBox(height: 16),

              // 3. Блок налаштування теми застосунку (п. 1, 2, 6 ТЗ)
              _buildThemeSwitchCard(context, isDark),

              const SizedBox(height: 16),

              // 4. Блок опцій профілю з власними іконками (п. 5, 6 ТЗ)
              _buildOptionsCard(context, isDark),

              const SizedBox(height: 20),

              // 5. Кнопка редагування профілю
              ElevatedButton.icon(
                style: ElevatedButton.styleFrom(
                  backgroundColor: AppColors.primary,
                  foregroundColor: Colors.white,
                  padding: const EdgeInsets.symmetric(vertical: 14),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
```

```
                icon: const Icon(Icons.edit_outlined, size: 20),
                label: const Text('Редагувати профіль', style: AppStyles.button),
                onPressed: () {
                  _showSnackBar(context, 'Режим редагування профілю активовано');
                },
              ),

              const SizedBox(height: 10),

              // 6. Кнопка виходу
              OutlinedButton.icon(
                style: OutlinedButton.styleFrom(
                  foregroundColor: AppColors.error,
                  side: const BorderSide(color: AppColors.error, width: 1.2),
                  padding: const EdgeInsets.symmetric(vertical: 14),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
                icon: const Icon(Icons.logout, size: 20),
                label: Text(
                  'Вийти з акаунта',
                  style: AppStyles.button.copyWith(color: AppColors.error),
                ),
                onPressed: () => _confirmLogout(context, isDark),
              ),
              const SizedBox(height: 16),
            ],
          ),
        ),
      ),
    );
  }

  /// Картка заголовка з банером, аватаром, ім'ям та статусом
  Widget _buildHeaderCard(BuildContext context, bool isDark) {
    return Container(
      decoration: BoxDecoration(
        color: AppColors.card(isDark),
        borderRadius: BorderRadius.circular(18),
        border: Border.all(color: AppColors.divider(isDark)),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: isDark ? 0.25 : 0.05),
            blurRadius: 10,
            offset: const Offset(0, 4),
          ),
        ],
      ),
      child: Column(
        children: [
          // Банерне зображення зверху (п. 4 ТЗ)
          ClipRRect(
            borderRadius: const BorderRadius.vertical(top: Radius.circular(17)),
            child: Image.asset(
              'assets/images/banner.png',
              height: 90,
              width: double.infinity,
              fit: BoxFit.cover,
            ),
          ),
          // Аватар із накладанням на банер
          Transform.translate(
            offset: const Offset(0, -42),
            child: Stack(
              alignment: Alignment.bottomRight,
              children: [
                Container(
```

```
                  padding: const EdgeInsets.all(4),
                  decoration: BoxDecoration(
                    color: AppColors.card(isDark),
                    shape: BoxShape.circle,
                    boxShadow: [
                      BoxShadow(
                        color: Colors.black.withValues(alpha: 0.15),
                        blurRadius: 8,
                      ),
                    ],
                  ),
                  child: const CircleAvatar(
                    radius: 46,
                    backgroundImage: AssetImage('assets/images/avatar.png'),
                  ),
                ),
                // Бейдж онлайн-статусу
                Container(
                  margin: const EdgeInsets.only(right: 6, bottom: 4),
                  width: 18,
                  height: 18,
                  decoration: BoxDecoration(
                    color: AppColors.success,
                    shape: BoxShape.circle,
                    border: Border.all(
                      color: AppColors.card(isDark),
                      width: 3,
                    ),
                  ),
                ),
              ],
            ),
          ),
          Transform.translate(
            offset: const Offset(0, -32),
            child: Padding(
              padding: const EdgeInsets.symmetric(horizontal: 16.0),
              child: Column(
                children: [
                  // Ім'я користувача (п. 6 ТЗ)
                  Text(
                    'Багрій-Белз Андрій',
                    textAlign: TextAlign.center,
                    style: AppStyles.title(isDark: isDark),
                  ),
                  const SizedBox(height: 4),
                  // Контактна інформація (email) (п. 6 ТЗ)
                  Row(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      const Icon(
                        Icons.email_outlined,
                        size: 15,
                        color: AppColors.primary,
                      ),
                      const SizedBox(width: 6),
                      Text(
                        'andrii.bagriy@nulp.ua',
                        style: AppStyles.subtitle(isDark: isDark),
                      ),
                    ],
                  ),
                  const SizedBox(height: 10),
                  // Короткий опис або статус (п. 6 ТЗ)
                  Container(
                    padding: const EdgeInsets.symmetric(
                      horizontal: 12,
                      vertical: 8,
```

```
                    ),
                    decoration: BoxDecoration(
                      color: AppColors.surface(isDark),
                      borderRadius: BorderRadius.circular(10),
                    ),
                    child: Text(
                      'Студент НУ «Львівська політехніка» | Група КН-2327Б\nРозробник мобільних
додатків на Flutter 🚀',
                      textAlign: TextAlign.center,
                      style: AppStyles.body(isDark: isDark).copyWith(fontSize: 13),
                    ),
                  ),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }

  /// Картка статистики та показників
  Widget _buildStatsCard(bool isDark) {
    return Container(
      padding: const EdgeInsets.symmetric(vertical: 14, horizontal: 8),
      decoration: BoxDecoration(
        color: AppColors.card(isDark),
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: AppColors.divider(isDark)),
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceAround,
        children: [
          _buildStatItem('Курс', '3-й', isDark),
          _buildDivider(isDark),
          _buildStatItem('Лабораторна', '№9', isDark, isHighlight: true),
          _buildDivider(isDark),
          _buildStatItem('Тема', isDark ? 'Dark' : 'Light', isDark),
        ],
      ),
    );
  }

  Widget _buildStatItem(String label, String value, bool isDark, {bool isHighlight = false}) {
    return Column(
      children: [
        Text(
          value,
          style: TextStyle(
            fontFamily: AppStyles.fontFamily,
            fontSize: 18,
            fontWeight: FontWeight.bold,
            color: isHighlight
                ? AppColors.primary
                : (isDark ? AppColors.textDark : AppColors.textLight),
          ),
        ),
        const SizedBox(height: 2),
        Text(
          label,
          style: AppStyles.caption(isDark: isDark),
        ),
      ],
    );
  }

  Widget _buildDivider(bool isDark) {
```

```
    return Container(
      width: 1,
      height: 28,
      color: AppColors.divider(isDark),
    );
  }

  /// Картка перемикання теми (п. 1, 2 ТЗ)
  Widget _buildThemeSwitchCard(BuildContext context, bool isDark) {
    return Material(
      color: AppColors.card(isDark),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
        side: BorderSide(color: AppColors.divider(isDark)),
      ),
      child: SwitchListTile(
        key: const Key('theme_switch_tile'),
        secondary: Container(
          padding: const EdgeInsets.all(6),
          decoration: BoxDecoration(
            color: (isDark ? AppColors.secondary : AppColors.primary)
                .withValues(alpha: 0.15),
            shape: BoxShape.circle,
          ),
          child: Image.asset(
            'assets/icons/theme.png',
            width: 24,
            height: 24,
          ),
        ),
        title: Text(
          'Темна тема (Dark Mode)',
          style: TextStyle(
            fontFamily: AppStyles.fontFamily,
            fontWeight: FontWeight.w600,
            fontSize: 15,
            color: isDark ? AppColors.textDark : AppColors.textLight,
          ),
        ),
        subtitle: Text(
          isDark ? 'Увімкнено нічний режим' : 'Увімкнено денний режим',
          style: AppStyles.caption(isDark: isDark),
        ),
        value: isDark,
        activeThumbColor: AppColors.secondary,
        onChanged: (bool value) {
          onToggleTheme(value);
        },
      ),
    );
  }

  /// Картка списку налаштувань та іконок оформлення (п. 5, 6 ТЗ)
  Widget _buildOptionsCard(BuildContext context, bool isDark) {
    final options = [
      {
        'iconPath': 'assets/icons/profile.png',
        'title': 'Персональні дані',
        'subtitle': 'Редагування імені, контактів та біографії',
      },
      {
        'iconPath': 'assets/icons/shield.png',
        'title': 'Безпека та конфіденційність',
        'subtitle': 'Пароль, двофакторна автентифікація',
      },
      {
        'iconPath': 'assets/icons/bell.png',
        'title': 'Сповіщення',
```

```
        'subtitle': 'Push-сповіщення, системні нагадування',
      },
      {
        'iconPath': 'assets/icons/settings.png',
        'title': 'Параметри застосунку',
        'subtitle': 'Мова інтерфейсу, синхронізація, кеш',
      },
    ];
    return Material(
      color: AppColors.card(isDark),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
        side: BorderSide(color: AppColors.divider(isDark)),
      ),
      child: ListView.separated(
        shrinkWrap: true,
        physics: const NeverScrollableScrollPhysics(),
        itemCount: options.length,
        separatorBuilder: (context, index) => Divider(
          color: AppColors.divider(isDark),
          height: 1,
          indent: 56,
        ),
        itemBuilder: (context, index) {
          final opt = options[index];
          return ListTile(
            leading: Image.asset(
              opt['iconPath']!,
              width: 28,
              height: 28,
            ),
            title: Text(
              opt['title']!,
              style: TextStyle(
                fontFamily: AppStyles.fontFamily,
                fontWeight: FontWeight.w600,
                fontSize: 14,
                color: isDark ? AppColors.textDark : AppColors.textLight,
              ),
            ),
            subtitle: Text(
              opt['subtitle']!,
              style: AppStyles.caption(isDark: isDark),
            ),
            trailing: Icon(
              Icons.chevron_right,
              color: isDark ? AppColors.textSecondaryDark : AppColors.grey,
              size: 20,
            ),
            onTap: () {
              _showSnackBar(context, 'Обрано: ${opt['title']}');
            },
          );
        },
      ),
    );
  }

  void _showSnackBar(BuildContext context, String message) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(
          message,
          style: const TextStyle(fontFamily: AppStyles.fontFamily),
        ),
        backgroundColor: isDarkMode ? AppColors.darkCard : AppColors.primary,
        behavior: SnackBarBehavior.floating,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
```

```
        duration: const Duration(seconds: 2),
      ),
    );
  }

  void _confirmLogout(BuildContext context, bool isDark) {
    showDialog(
      context: context,
      builder: (ctx) => AlertDialog(
        backgroundColor: AppColors.card(isDark),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
        title: Text(
          'Вихід із профілю',
          style: AppStyles.title(isDark: isDark).copyWith(fontSize: 18),
        ),
        content: Text(
          'Ви впевнені, що бажаєте вийти зі свого облікового запису?',
          style: AppStyles.body(isDark: isDark),
        ),
        actions: [
          TextButton(
            child: Text(
              'Скасувати',
              style: TextStyle(
                color: isDark
                    ? AppColors.textSecondaryDark
                    : AppColors.textSecondaryLight,
              ),
            ),
            onPressed: () => Navigator.of(ctx).pop(),
          ),
          ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: AppColors.error,
              foregroundColor: Colors.white,
            ),
            child: const Text('Вийти'),
            onPressed: () {
              Navigator.of(ctx).pop();
              _showSnackBar(context, 'Ви успішно вийшли з облікового запису');
            },
          ),
        ],
      ),
    );
  }
}
```

Лістинг: test/widget_test.dart (Автоматизовані віджет-тести)

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:profile_app/main.dart';
import 'package:profile_app/theme/app_colors.dart';
import 'package:profile_app/theme/app_styles.dart';

void main() {
  testWidgets('1. Відображення екрана ProfileScreen та обов’язкових елементів', (
    WidgetTester tester,
  ) async {
    await tester.pumpWidget(const ProfileApp());
    await tester.pumpAndSettle();

    // Перевірка наявності AppBar та імені користувача
    expect(find.text('Профіль користувача'), findsOneWidget);
```

```
16      expect(find.text('Багрій-Белз Андрій'), findsOneWidget);
17
18      // Перевірка email та статусу
19      expect(find.text('andrii.bagriy@nulp.ua'), findsOneWidget);
20      expect(
21        find.textContaining('Студент НУ «Львівська політехніка»'),
22        findsOneWidget,
23      );
24
25      // Перевірка наявності перемикача теми
26      expect(find.text('Темна тема (Dark Mode)'), findsOneWidget);
27      expect(find.byKey(const Key('theme_switch_tile')), findsOneWidget);
28
29      // Перевірка наявності кнопок
30      expect(find.text('Редагувати профіль'), findsOneWidget);
31      expect(find.text('Вийти з акаунта'), findsOneWidget);
32    });
33
34    testWidgets('2. Перемикання теми з темної (Dark) на світлу (Light)', (
35      WidgetTester tester,
36    ) async {
37      await tester.pumpWidget(const ProfileApp());
38      await tester.pumpAndSettle();
39
40      // За замовчуванням темний режим
41      expect(find.text('Увімкнено нічний режим'), findsOneWidget);
42
43      // Натискаємо на SwitchListTile для перемикання на світлу тему
44      await tester.tap(find.byKey(const Key('theme_switch_tile')));
45      await tester.pumpAndSettle();
46
47      // Стан повинен оновитися на денний режим
48      expect(find.text('Увімкнено денний режим'), findsOneWidget);
49    });
50
51    testWidgets('3. Повторне перемикання теми зі світлої назад у темну', (
52      WidgetTester tester,
53    ) async {
54      await tester.pumpWidget(const ProfileApp());
55      await tester.pumpAndSettle();
56
57      // Перше перемикання: Dark -> Light
58      await tester.tap(find.byKey(const Key('theme_switch_tile')));
59      await tester.pumpAndSettle();
60      expect(find.text('Увімкнено денний режим'), findsOneWidget);
61
62      // Друге перемикання: Light -> Dark
63      await tester.tap(find.byKey(const Key('theme_switch_tile')));
64      await tester.pumpAndSettle();
65      expect(find.text('Увімкнено нічний режим'), findsOneWidget);
66    });
67
68    testWidgets('4. Перевірка цілісності палітри AppColors та стилів AppStyles', (
69      WidgetTester tester,
70    ) async {
71      // Перевірка узгодженості кольорів
72      expect(AppColors.primary, const Color(0xFF4A90E2));
73      expect(AppColors.secondary, const Color(0xFFF5A623));
74      expect(AppColors.lightBackground, isNot(equals(AppColors.darkBackground)));
75      expect(AppColors.textLight, isNot(equals(AppColors.textDark)));
76
77      // Перевірка налаштування шрифту
78      expect(AppStyles.fontFamily, 'Inter');
79      expect(AppStyles.titleStatic.fontFamily, 'Inter');
80      expect(AppStyles.button.fontFamily, 'Inter');
81    });
82
83    testWidgets('5. Відображення діалогу підтвердження виходу з акаунта', (
```

```
    WidgetTester tester,
  ) async {
    await tester.pumpWidget(const ProfileApp());
    await tester.pumpAndSettle();

    // Знаходимо та натискаємо кнопку виходу
    final logoutButton = find.text('Вийти з акаунта');
    await tester.ensureVisible(logoutButton);
    await tester.tap(logoutButton);
    await tester.pumpAndSettle();

    // Перевіряємо відображення діалогового вікна AlertDialog
    expect(find.byType(AlertDialog), findsOneWidget);
    expect(find.text('Вихід із профілю'), findsOneWidget);
    expect(find.text('Скасувати'), findsOneWidget);

    // Скасовуємо вихід
    await tester.tap(find.text('Скасувати'));
    await tester.pumpAndSettle();

    expect(find.byType(AlertDialog), findsNothing);
  });
}
```

4. Результати виконання та тестування програми

Застосунок було зібрано та верифіковано в середовищі емулятора Android (API 36, Android 16), а також перевірено автоматизованим тестовим набором flutter test.

Тестування: Результати автоматизованого тестування (5/5 тестів пройдено успішно):

Лістинг: Результат виконання flutter test

```
00:00 +0: loading test/widget_test.dart
00:00 +0: 1. Відображення екрана ProfileScreen та обов’язкових елементів
00:00 +1: 2. Перемикання теми з темної (Dark) на світлу (Light)
00:00 +2: 3. Повторне перемикання теми зі світлої назад у темну
00:00 +3: 4. Перевірка цілісності палітри AppColors та стилів AppStyles
00:00 +4: 5. Відображення діалогу підтвердження виходу з акаунта
00:00 +5: All tests passed!
```

Демонстрація перемикання тем (Dark Mode vs Light Mode)

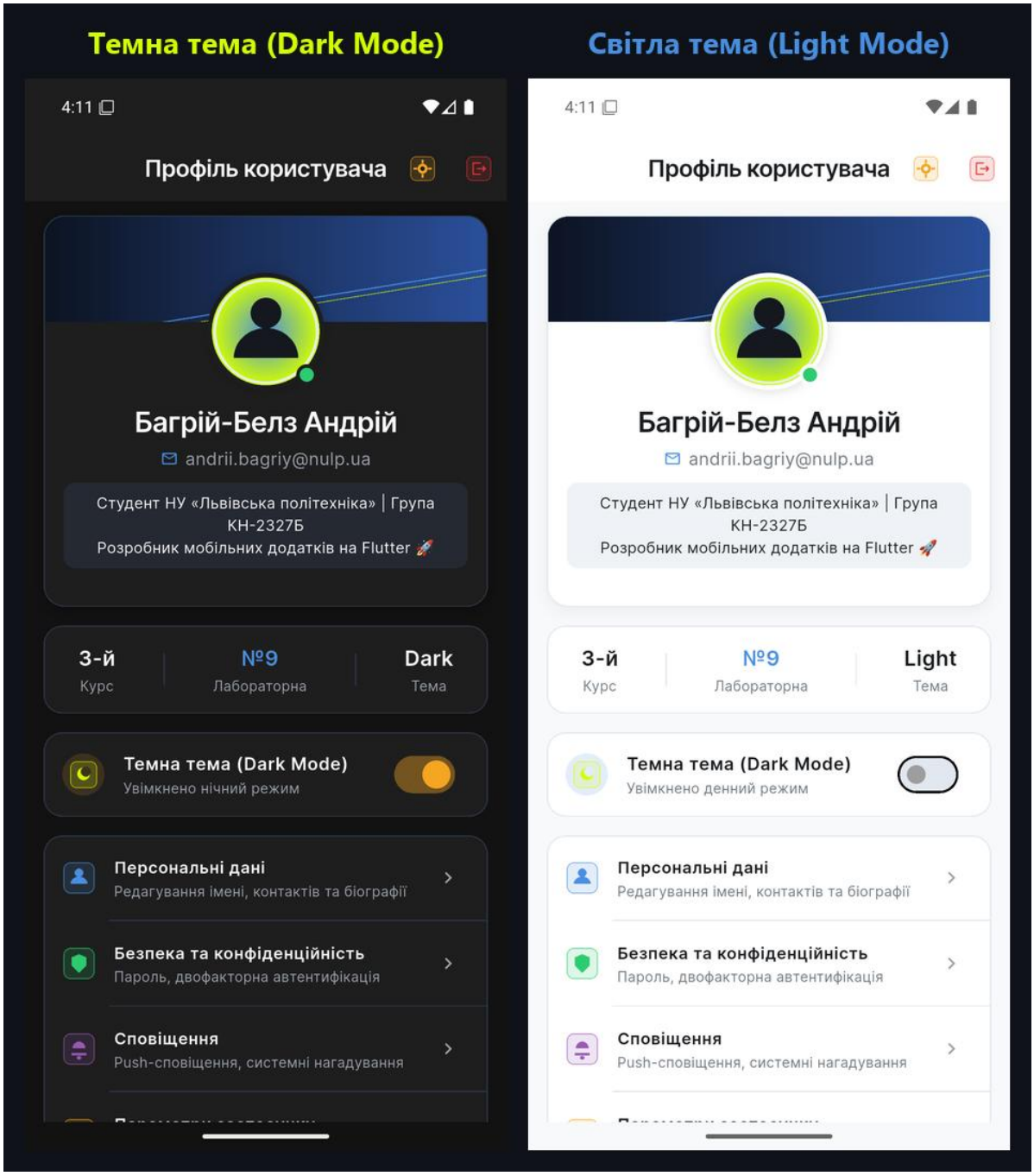
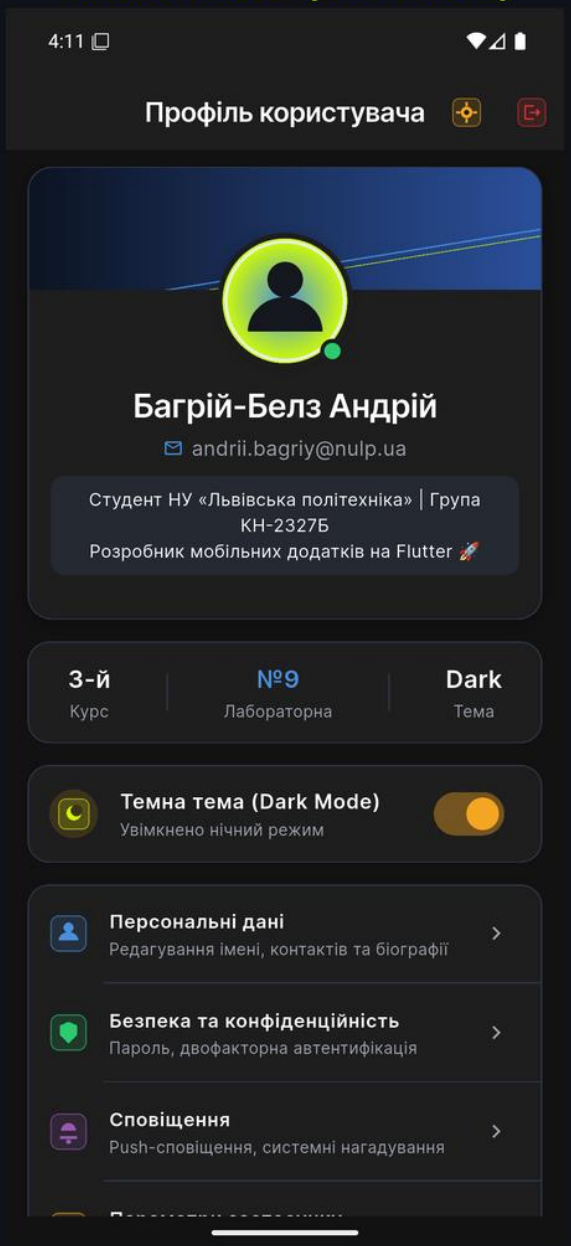


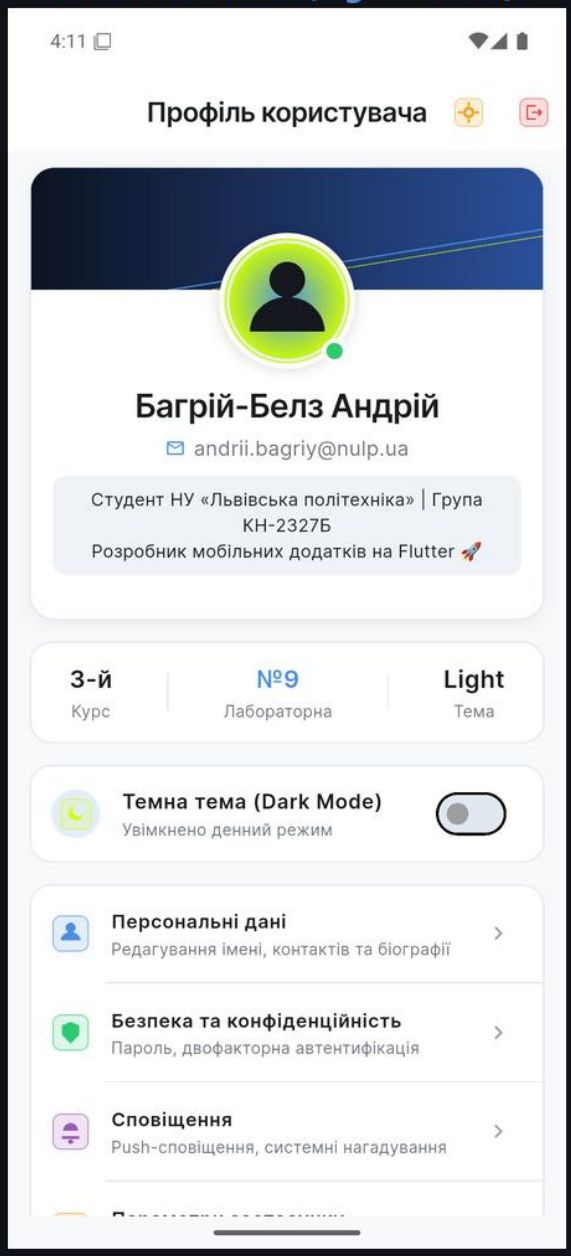
Рис. 9.1 — Порівняння екрана профілю у темній (Dark) та світлій (Light) темах

Повний цикл роботи застосунку (опції, кнопки та діалог виходу)

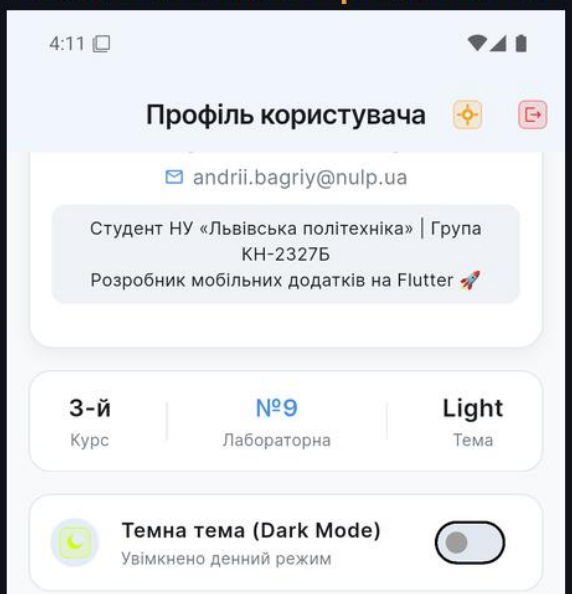
1. Темна тема (Dark Mode)



2. Світла тема (Light Mode)



3. Світла тема: опції та кнопки



4. Діалогове вікно підтвердження

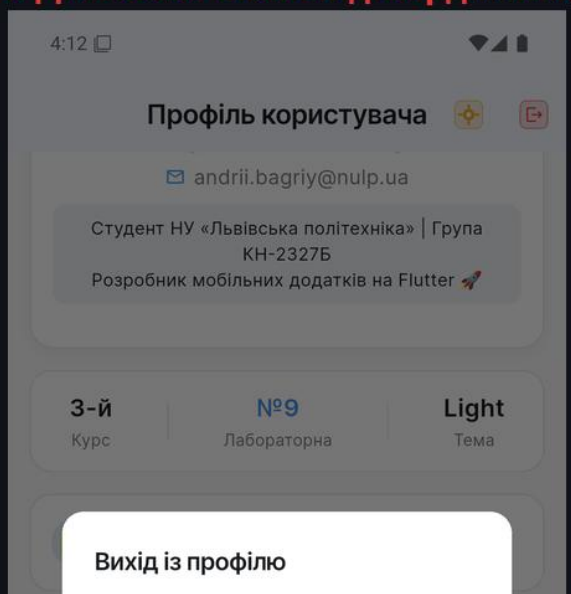


Рис. 9.2 — Екрани профілю: Dark Mode, Light Mode, прокручений список опцій та діалог виходу

5. Відповіді на контрольні запитання

1. Що таке ThemeData у Flutter?

ThemeData — це клас конфігурації у Flutter SDK, який визначає глобальні візуальні та типографічні властивості інтерфейсу застосунку або окремої гілки дерева віджетів. Він агрегує повний набір стилістичних параметрів: колірну схему (ColorScheme), типографіку (TextTheme), кольори поверхонь та фону (scaffoldBackgroundColor), стилі окремих компонентів (AppBarTheme, CardTheme, ElevatedButtonThemeData, SwitchThemeData тощо), властивості відступів, округлень та тіней. Через механізм Theme.of(context) дочірні віджети автоматично адаптують свій вигляд до налаштувань поточної теми за принципом наслідування віджетів (InheritedWidget), усуваючи потребу вручну передавати параметри оформлення.

2. Для чого використовуються theme, darkTheme та themeMode?

У кореневому віджеті MaterialApp:
● theme — задає об'єкт ThemeData для світлого режиму оформлення (Light mode). Використовується за замовчуванням або коли активний світлий режим;
● darkTheme — задає об'єкт ThemeData для темного режиму оформлення (Dark mode). Автоматично застосовується системою, коли пристрій переходить у нічний режим або коли явно обрано темну тему;
● themeMode — визначає політику вибору активної теми серед значень enum ThemeMode: ThemeMode.system (адаптація до налаштувань ОС), ThemeMode.light (примусова світла тема), ThemeMode.dark (примусова темна тема).

3. Як реалізувати перемикання світлої і темної теми у застосунку?

Динамічне перемикання теми реалізується шляхом керування станом активного режиму (наприклад, булевою змінною bool isDarkMode або екземпляром ThemeMode):
1. У кореневому віджеті зі станом (StatefulWidget) або через менеджер стану (Provider, Riverpod, ValueNotifier) зберігається змінна поточного стану теми;
2. Властивість themeMode у MaterialApp зв'язується з цим станом: themeMode: isDarkMode ? ThemeMode.dark : ThemeMode.light;
3. У налаштуваннях або профілі користувача розміщується перемикач (Switch або SwitchListTile), обробник onChanged якого викликає setState(() => isDarkMode = value) або сповіщає слухачів;
4. Після зміни стану кореневий віджет перебудовується, і MaterialApp плавно змінює глобальну тему застосунку.

4. Що таке ресурси (assets) у Flutter? Які їх основні типи?

Ресурси (assets) — це статичні зовнішні файли, які пакуються разом із виконуваним бінарним файлом застосунку (APK, IPA або вебархівом) і доступні під час виконання програми через клас AssetBundle без звернення до мережі.
Основні типи ресурсів:
● Растрові графічні зображення: PNG, JPEG, GIF, WebP (фотографії, банери, аватари);

● Векторні зображення: SVG (відображаються за допомогою пакету flutter_svg);
● Шрифти (Fonts): файли форматів TTF, OTF для кастомної типографіки та брендових накреслень;
● Іконки: растрові PNG-іконки або векторні кастомні icon-фонти;
● Файли конфігурації та даних: JSON, YAML, текстові файли, локалізаційні словники (ARB);
● Медіа-файли: анімації Lottie (JSON), векторні сцени Rive, аудіо та відео.

5. Як додати зображення та шрифти до Flutter-проєкту?

Процес додавання ресурсів складається з двох послідовних кроків:
1. Фізичне розміщення файлів: створення каталогів у корені проєкту (наприклад, assets/images/, assets/fonts/, assets/icons/) та копіювання туди файлів (avatar.png, Inter-Regular.ttf);
2. Реєстрація у файлі конфігурації pubspec.yaml:

```
flutter:
  assets:
    - assets/images/
    - assets/icons/
  fonts:
    - family: Inter
      fonts:
        - asset: assets/fonts/Inter-Regular.ttf
        - asset: assets/fonts/Inter-SemiBold.ttf
          weight: 600
```

Після реєстрації виконується команда flutter pub get, після чого ресурси стають доступними через AssetImage('assets/images/avatar.png') або властивість fontFamily: 'Inter'.

6. Які типи іконок можна використовувати у Flutter?

У Flutter можна застосовувати такі різновиди іконок:
● Стандартні іконки SDK: віджети Icon(Icons.name) на основі шрифту Material Icons або CupertinoIcons. Вони є векторними, ідеально масштабуються без втрати чіткості та легко стилізуються (color, size);
● Растрові іконки: зображення форматів PNG або WebP, що відображаються через Image.asset('assets/icons/name.png');
● Векторні SVG-іконки: файли .svg, які рендеряться бібліотекою flutter_svg (SvgPicture.asset). Забезпечують ідеальну чіткість при будь-якій щільності пікселів екрана;
● Власні шрифтові іконки (IconFont): власноруч згенеровані шрифти через сервіси FlutterIcon, де символи мапляться на IconData.

7. Що таке AppColors і для чого він використовується?

AppColors — це спеціалізований клас-палітра (шар дизайн-токенів), призначений для централізованого збереження та семантичного іменування всіх кольорів застосунку. Замість хаотичного використання розрізнених значень (Colors.blue чи Color(0xFF...)) він надає чіткі константи: primary, secondary, lightBackground, darkBackground, lightCard, textLight тощо.
Він використовується для:

● Забезпечення єдиної візуальної дизайн-системи у застосунку;
● Швидкого редизайну або зміни бренд-палітри з одного місця;
● Зручної підтримки темного та світлого режимів завдяки динамічним селекторам (AppColors.background(isDark));
● Підвищення читабельності та чистоти коду без дублювання hex-значень.

8. Чому не рекомендується використовувати кольори напряму (Colors.blue) у проєкті?

Пряме використання констант на кшталт Colors.blue, Colors.black або Colors.white є антипатерном з кількох суттєвих причин:
● Відсутність єдиної точки модифікації: будь-яка зміна фірмового кольору бренду призведе до необхідності рефакторингу десятків файлів по всьому проєкту;
● Складність підтримки темної теми: прямі кольори не адаптуються до зміни контрастності, що спричиняє погану читабельність (наприклад, темний текст на темному фоні);
● Порушення візуальної консистентності: різні розробники можуть обрати несумісні відтінки (Colors.blue, Colors.lightBlue, Colors.blueAccent), що руйнує цілісність інтерфейсу;
● Неможливість впровадження динамічного темування (White-labeling): жорстко закодовані кольори не дозволяють адаптувати продукт під різні корпоративні стилі або пресети доступності.

9. Як використовувати власні кольори у ThemeData та віджетах?

● У ThemeData: кольори з AppColors передаються у конструктор теми через colorScheme: ColorScheme.light(primary: AppColors.primary, surface: AppColors.lightCard...), scaffoldBackgroundColor: AppColors.lightBackground, cardTheme: CardThemeData(color: AppColors.lightCard), appBarTheme тощо.
● У віджетах: рекомендовано отримувати кольори або семантично через Theme.of(context).colorScheme.primary, або безпосередньо через статичні методи палітри: AppColors.card(isDark), AppColors.text(isDark). Це гарантує, що компонент завжди відображатиме коректний колір відповідно до поточної теми.

10. Що таке AppStyles і яка його роль у проєкті?

AppStyles — це клас для централізованої стандартизації типографіки застосунку. Він інкапсулює готові стилі тексту (TextStyle) для різних рівнів візуальної ієрархії: заголовків (title), підзаголовків (subtitle), основного тексту (body), написів кнопок (button), підписів (caption) та бейджів (badge).
Його роль у проєкті:
● Гарантування єдиного шрифту (fontFamily: 'Inter'), розмірів кегля, товщини (fontWeight) та міжрядкових інтервалів (height) у всіх компонентах;
● Усунення ручного дублювання об'єктів TextStyle у верстці;
● Підтримка швидкої адаптації контрастності тексту під світлу та темну теми (AppStyles.title(isDark: isDark));
● Спрощення підтримки та масштабування дизайну застосунку.

11. Які основні елементи повинен містити екран профілю?

Повноцінний екран профілю користувача містить такі ключові елементи:
1. Аватар користувача: фотографія або графічне зображення користувача (CircleAvatar) з індикатором онлайн-статусу або кнопкою редагування;
2. Ідентифікаційні дані: повне ім'я користувача (ПІБ або нікнейм) та контактні відомості (електронна пошта, номер телефону);
3. Короткий опис / статус (Bio): текстова плашка з посадою, місцем навчання або персональним статусом;
4. Блок статистики або досягнень: показники активності (курс, група, кількість проєктів, виконані завдання);
5. Елементи налаштувань та керування темою: інтерактивний перемикач теми (SwitchListTile для Light / Dark mode);
6. Навігаційні розділи з іконками: пункти меню (персональні дані, безпека та паролі, сповіщення, параметри застосунку);
7. Кнопки дій: кнопки редагування профілю та виходу з облікового запису з діалогом підтвердження.

12. Як забезпечити єдиний стиль оформлення застосунку?

Забезпечення єдиного стилю (консистентності) досягається дотриманням комплексу архітектурних та дизайнерських правил:
● Централізація кольорів (Design Tokens): створення єдиного класу AppColors та сувора заборона прямого використання сторонніх кольорів;
● Централізація типографіки: використання єдиного сімейства шрифтів (Inter) через AppStyles та параметри ThemeData.textTheme;
● Системна конфігурація тем: опис світлої та темної тем через ThemeData (AppTheme.lightTheme та AppTheme.darkTheme);
● Стандартизація просторової сітки (Spacing System): використання узгоджених кроків відступів (SizedBox(height: 8/16/24)), радіусів скруглення кутів карток (BorderRadius.circular(12/16)) та товщини меж;
● Єдині компоненти та піктограми: використання уніфікованого набору іконок однакового візуального стилю та стилізованих карток для групування контенту.

6. Висновок

Під час виконання лабораторної роботи №9 було успішно досліджено та практично реалізовано механізми оформлення мобільного Flutter-застосунку з використанням тем (ThemeData), централізованих дизайн-токенів (AppColors, AppStyles, AppTheme), підключено власний шрифт сімейства Inter через pubspec.yaml, а також інтегровано локальні растрові графічні ресурси (банерне зображення, аватар користувача та комплект тематичних PNG-іконок). Розроблено інтерактивний екран профілю користувача ProfileScreen з повною інформаційною структурою (аватар, статус онлайн, контактні дані, біографія, блок академічної статистики, розділи налаштувань, інтерактивні кнопки редагування та виходу з діалогом підтвердження). Реалізовано безшовне перемикання теми (Light / Dark mode) за допомогою віджета SwitchListTile та динамічного оновлення ThemeMode у MaterialApp. Працездатність застосунку,

узгодженість кольорової гами та типографіки у світлому і темному режимах повністю підтверджено ручним тестуванням на віртуальному пристрої Android (API 36) та повним набором автоматизованих віджет-тестів.